SatsPath Resolver Semantics v1

Resolver provenance and verification are reported separately. A transparent result carries the signed profile, latest name event, inclusion proof, signed checkpoint, optional consistency proof and identifier attestation. Implementations expose independent states for profile signature, identifier binding, key continuity, log inclusion, checkpoint consistency and payment-method ownership; these must not be collapsed into a single `verified` boolean.

SatsPath is transport-neutral. A resolver is any component that can map a SatsPath identifier to a `SignedPaymentProfile`.

Resolvers do not decide whether money moves. Resolvers only discover signed profile data. The quote pipeline verifies and routes after resolution.

Resolver Interface

Conceptually, every resolver implements:

```
resolve(identifier) -> SignedPaymentProfile | NotFound | Unavailable | Invalid
```

The Rust reference implementation exposes this as `ProfileResolver`:

```rust
async fn resolve_alias(&self, alias: &str) -> Result<SignedPaymentProfile>;
```

`AliasNotFound` maps to `NotFound`. Network or transport errors map to `Unavailable` unless the resolver can prove the returned data is authoritative and invalid.

Resolver Chain

The default resolver chain is:

```
local registry -> BIP-353 -> HTTPS -> Nostr
```

Implementations MAY add other resolvers, including P2P transports, platform APIs, QR-imported profiles, or hardware-wallet-provided profile stores.

The chain MUST:

1. Normalize the identifier.
2. Query each resolver in order.
3. Continue past `NotFound`.
4. Continue past transient `Unavailable` failures.
5. Verify any returned signed profile before routing.
6. Stop at the first verified profile.
7. Return `not_registered` if no resolver returns a usable profile.

The chain MUST NOT trust a resolver because of its transport. An HTTPS response, DNS response, Nostr event, Pear/Holepunch message, or local file all require the same signature verification.

Identifier Verification

SatsPath separates profile verification from identifier verification:

- Profile verification means the `SignedPaymentProfile` signature verifies against `profile.identity_pubkey`.
- Identifier verification means an external authority or challenge proves that the canonical identifier was controlled or approved for that profile.

Plain email syntax, such as `alice@gmail.com`, is only an identifier. It does not prove inbox access, domain ownership, or payment-method ownership.

DNSSEC/BIP-353 and Nostr/NIP-05 can verify publication for domains the receiver controls. Consumer email domains usually cannot publish records under their own domain, so those identifiers require an explicit platform challenge resolver or fall back to the invite flow.

The daemon sets `identifier_verified` only when an attestation binds the exact identifier hash, key and profile hash in the latest event and its verifier key and method are explicitly trusted. Otherwise it remains false and first contact is TOFU.

Local Registry Resolver

The local registry is a file-backed resolver used by CLI and daemon flows.

Reference files:

- `.satspath/satspath-transparency-v1.sqlite3` (daemon security state)
- `.satspath/registry.json` (legacy CLI/local resolver compatibility only)
- `.satspath/peers/registry.local.json`

The local registry MAY store profiles created locally or imported from another transport. A profile imported into the registry MUST be verified before use.

BIP-353 DNS Resolver

BIP-353 resolves a Bitcoin payment instruction from DNS TXT records. In SatsPath v1, BIP-353 can provide:

- A direct `bitcoin:` / BIP-321 payment instruction.
- A pointer to a SatsPath signed profile, such as `sp-profile`.
- A hash commitment, such as `sp-profile-hash`.

Strict DNS behavior:

- DNSSEC validation is required for trustworthy mainnet use.
- If no DNSSEC-validating resolver is available, the implementation MUST fail closed.
- Development mode MAY accept unvalidated DNS only behind an explicit flag.

The reference implementation provides:

- `DohTxtResolver`
- `resolve_bip353_with`
- `DnssecPolicy::Strict`
- `DnssecPolicy::DevInsecure`

HTTPS Resolver

The HTTPS resolver maps an identifier to a well-known signed profile endpoint.

Expected endpoint shape:

`https://<domain>/.well-known/satspath/<user>`

The resolver MUST:

- Fetch JSON over HTTPS.
- Decode as `SignedPaymentProfile`.
- Verify the profile signature.
- Reject malformed JSON.
- Treat HTTP 404 as `NotFound`.
- Treat transport failures as `Unavailable`.

HTTP transport security does not replace profile signatures.

Nostr Resolver

The Nostr resolver is a transport for discovering signed profile data through Nostr events. It does not require Pear/Holepunch or any SatsPath-specific P2P network.

Resolution flow:

1. Parse `user@domain`.
2. Fetch `https://domain/.well-known/nostr.json?name=user`.
3. Read the NIP-05 pubkey and relay hints.
4. Query relays for a Nostr kind `30078` event authored by that pubkey.
5. Require a `d` tag equal to `satspath-profile:<canonical user@domain>`.
6. Parse the event `content` as `SignedPaymentProfile`.
7. Verify the SatsPath profile signature and expiry.

Nostr event signatures and SatsPath profile signatures are separate layers. The Nostr event proves which Nostr key published the event. The SatsPath signature proves that the payment profile is controlled by the SatsPath protocol identity key.

The resolver MUST still return a `SignedPaymentProfile` and the quote pipeline MUST verify the SatsPath profile signature.

NIP-05 verifies that the domain's well-known metadata maps the local identifier part to a Nostr pubkey. It does not verify a consumer email inbox unless that same domain/account infrastructure explicitly performs such verification.

Reference event shape:

```
{
  "kind": 30078,
  "pubkey": "<nip05 nostr pubkey hex>",
  "tags": [
    ["d", "satspath-profile:alice@example.com"]
  ],
  "content": "{\"profile\":{...},\"signature\":\"3044...\"}"
}
```

Implementations MAY configure fallback relays. The Rust implementation reads `SATSPATH_NOSTR_RELAYS` as a comma-separated list and otherwise uses default public relays.

P2P Resolver

Pear/Holepunch P2P transport is optional. It can announce, request, and return signed profiles between peers.

P2P discovery MUST NOT be treated as weaker or stronger than HTTPS or DNS by default. The returned profile is still untrusted until:

1. The response decodes as `SignedPaymentProfile`.
2. The SatsPath profile signature verifies.
3. Expiry checks pass.
4. Optional method ownership checks pass.

P2P wire behavior is specified in wire_p2p.md.

Resolver Output and Quote Mapping

Resolver outcome maps to quote behavior as follows:

Resolver outcome	Quote behavior
Profile plus all required transparency evidence verified	Continue with ownership-verified methods only
Transparency, checkpoint binding or operator continuity fails	`no_route` / explicit transparency error; never route
No resolver found a profile	`not_registered`
Profile found but signature invalid	`invalid_signature`
Profile expired	`no_route`
Profile has no supported rail	`no_route`
Transient resolver failures only	`not_registered` unless a stricter caller wants diagnostics

Implementations SHOULD expose resolver diagnostics in debug APIs, but the user-facing quote response remains the stable protocol contract.

Conformance

A SatsPath resolver implementation is conformant if it:

- Returns signed profile data, not wallet secrets.
- Does not decide payment execution.
- Does not skip profile verification.
- Distinguishes not found from invalid data.
- Can be composed in a resolver chain.
- Produces data compatible with the `QuoteResponse` pipeline.